

OpenTofu + libvirt

Orquestación de infraestructura con KVM

✉ José Domingo Muñoz

🏠 IES Gonzalo Nazareno · Dos Hermanas

📖 PI · Puesta en Producción de Aplicaciones

PI · OPENTOFU + LIBVIRT

01

Terraform y OpenTofu

Historia, licencia y por qué usamos OpenTofu

¿Qué es Terraform?

TERRAFORM

- Herramienta de **orquestación de infraestructura** creada por **HashiCorp** en 2014
- Lenguaje declarativo propio: **HCL** (*HashiCorp Configuration Language*)
- Permite definir, crear y administrar infraestructura como **código**
- Compatible con múltiples proveedores: AWS, Azure, GCP, OpenStack, libvirt...

MODELO DECLARATIVO

En lugar de describir *cómo* crear la infraestructura, **declaramos el estado deseado** y la herramienta se encarga de alcanzarlo.

```
resource "libvirt_domain" "server1" {  
  name    = "servidor1"  
  memory = 1024  
  vcpu    = 1  
}
```

El problema con Terraform: cambio de licencia

AGOSTO 2023 — HASHICORP CAMBIA LA LICENCIA

- Terraform pasa de **MPL 2.0** (libre) a **BUSL 1.1** (*Business Source License*)
- BUSL **no es software libre**: prohíbe usar Terraform para ofrecer servicios que compitan con HashiCorp
- La comunidad reacciona con rechazo

CONSECUENCIAS

- Los usuarios no pueden garantizar el uso sin restricciones en el futuro
- Proyectos que dependen de Terraform quedan en una zona gris legal
- Se hace necesaria una alternativa **100 % libre y compatible**

¿Qué es OpenTofu?

" *OpenTofu es un fork de **Terraform 1.5.x**, creado por la comunidad (liderado por la Linux Foundation) como respuesta al cambio de licencia de Terraform en 2023.*

LICENCIA LIBRE

Licenciado bajo **MPL 2.0**
(*Mozilla Public License*),
reconocida como licencia
libre y open source

GOBERNANZA ABIERTA

Gobernado por la **Linux Foundation**. Todo el
desarrollo es abierto y las
decisiones se discuten
públicamente

COMPATIBILIDAD TOTAL

Compatible con los ficheros
`.tf` y los *providers*
existentes de Terraform. Sin
restricciones de uso

¿Por qué usamos OpenTofu y no Terraform?

OPENTOFU

- Licencia **MPL 2.0** — 100 % libre
- Sin restricciones de uso en entornos educativos ni comerciales
- Gobernado por la **Linux Foundation**
- Desarrollo transparente y comunitario
- Compatible con todo el ecosistema de Terraform

TERRAFORM

- Licencia **BUSL 1.1** — no libre
- Restricciones de uso para productos o servicios competidores
- Controlado únicamente por **HashiCorp** (ahora IBM)
- Puede cambiar las condiciones en cualquier momento



OpenTofu conserva el **modelo declarativo de IaC** y es totalmente compatible con los ficheros `.tf` y los *providers* existentes.

PI · OPENTOFU + LIBVIRT

02

Conceptos básicos de OpenTofu

Providers, resources, variables y outputs

¿Qué hace OpenTofu?

" *Permite **definir, crear y administrar infraestructura** (máquinas virtuales, redes, contenedores, etc.) usando archivos de texto declarativos (`.tf`).*

QUÉ DESCRIBES TÚ

```
resource "libvirt_domain" "server1" {  
  name    = "maquina1"  
  memory = 1024  
  vcpu    = 1  
}
```

Qué infraestructura quieres — sin detallar los pasos

QUÉ HACE OPENTOFU

- **Construye** la infraestructura si no existe
- **Modifica** los recursos que hayan cambiado
- **Destruye** los recursos que ya no estén declarados

De forma **reproducible y automatizada**

¿Qué es un *provider*?

" Un **provider** (proveedor) es un **plugin** que permite a OpenTofu interactuar con una plataforma o tecnología concreta. Cada provider sabe cómo crear, leer, actualizar y eliminar recursos.

EJEMPLOS DE PROVIDERS

PROVIDER	PLATAFORMA
aws	Amazon Web Services
azurerm	Microsoft Azure
google	Google Cloud Platform
libvirt	Máquinas virtuales con KVM
docker	Contenedores Docker

PROVIDER LIBVIRT

El provider `libvirt` permite a OpenTofu **crear y gestionar VMs en tu sistema KVM local**.

```
terraform {
  required_providers {
    libvirt = {
      source = "dmacvicar/libvirt"
    }
  }
}
```

Ficheros de configuración .tf

FICHEROS HABITUALES EN UN PROYECTO

- **provider.tf** — configura el provider a usar.
Se ejecuta `tofu init` una sola vez
- **variables.tf** — declara variables globales reutilizables
- **main.tf** — define los recursos: VMs, redes, discos...
- **output.tf** — muestra información al finalizar (IPs, nombres...)
- **networks.tf** — define las redes (si se gestionan con OpenTofu)

EJEMPLO: VARIABLE Y OUTPUT

```
variable "libvirt_pool_name" {  
    default = "default"  
}  
  
output "server1_ip" {  
    value = libvirt_domain.server1  
        .network_interface[0].addresses[0]  
}
```

PI · OPENTOFU + LIBVIRT

03

Comandos de OpenTofu

El flujo de trabajo básico

Comandos más importantes

COMANDO	DESCRIPCIÓN
<code>tofu init</code>	Inicializa el proyecto, descarga los <i>providers</i> y prepara el entorno
<code>tofu plan</code>	Muestra qué acciones realizará OpenTofu (sin aplicar cambios)
<code>tofu apply</code>	Aplica los cambios: crea, modifica o elimina recursos según los <code>.tf</code>
<code>tofu destroy</code>	Elimina todos los recursos gestionados por el proyecto
<code>tofu validate</code>	Verifica que la sintaxis de los archivos <code>.tf</code> sea correcta
<code>tofu show</code>	Muestra el estado actual de los recursos creados
<code>tofu output</code>	Muestra los valores definidos en bloques <code>output</code>

Flujo de trabajo habitual

PRIMERA VEZ

```
# 1. Inicializar – solo una vez
tofu init

# 2. Revisar los cambios
tofu plan

# 3. Aplicar el escenario
tofu apply
```

Aparecerá la confirmación de las acciones.
Escribe para continuar.

GESTIÓN DEL ESCENARIO

```
# Ver la información del output
tofu output

# Ver el estado de los recursos
tofu show

# Destruir todo el escenario
tofu destroy
```

PI · OPENTOFU + LIBVIRT

04

OpenTofu + libvirt

Creación de VMs con cloud-init

Imágenes cloud y cloud-init

IMÁGENES CLOUD

- Plantillas de sistemas operativos preparadas para aprovisionamiento automático
- Tamaño muy reducido (sin escritorio, configuración mínima)
- Formato `qcow2`

```
# Descargar imágenes base
sudo wget https://cloud.debian.org/images/cloud/...
        -O debian13-base.qcow2
sudo wget https://cloud-images.ubuntu.com/...
        -O ubuntu2404-base.qcow2
```

CLOUD-INIT

Herramienta que configura automáticamente la máquina en su **primer arranque**:

- Hostname
- Usuario y contraseña / clave SSH
- Paquetes a instalar
- Configuración de red (netplan)

OpenTofu genera un **disco ISO** con la configuración cloud-init.

Recursos típicos en `main.tf`

```
# Disco principal – clonación enlazada desde la imagen base
resource "libvirt_volume" "server1-disk" {
  name          = "server1.qcow2"
  pool          = var.libvirt_pool_name
  base_volume_id = "/var/lib/libvirt/images/debian13-base.qcow2"
  format        = "qcow2"
}
```

```
# Disco ISO con la configuración cloud-init
resource "libvirt_cloudinit_disk" "server1-cloudinit" {
  name      = "server1-cloudinit.iso"
  user_data = data.template_file.user_data.rendered
}
```

```
# Máquina virtual
resource "libvirt_domain" "server1" {
  name     = "server1"
  memory   = 1024
  vcpu     = 1
  disk     = [{ volume_id = libvirt_volume.server1-disk.id }, ...]
  cloudinit = [libvirt_cloudinit_disk.server1-cloudinit, ...]
```


Tipos de red con OpenTofu + libvirt

REDES NO GESTIONADAS POR OPENTOFU

Se referencia por nombre:

```
network_interface {  
  network_name    = "default"  
  wait_for_lease  = true  
}
```

REDES GESTIONADAS POR OPENTOFU

Se crea y se referencia por ID:

```
resource "libvirt_network" "nat_dhcp" {  
  name      = "red-nat"  
  mode      = "nat"  
  addresses = ["192.168.100.0/24"]  
}  
  
network_interface {  
  network_id      = libvirt_network.nat_dhcp.id  
  wait_for_lease  = true  
}
```

PI · OPENTOFU + LIBVIRT

05

Módulos de OpenTofu

Reutilización y organización de escenarios

¿Qué es un módulo?

" *Un **módulo** es un conjunto de archivos de configuración que agrupa recursos relacionados, de forma que puedan **reutilizarse, organizarse y parametrizarse** fácilmente.*

REUTILIZACIÓN

Define la infraestructura una sola vez y úsala en **múltiples entornos**: desarrollo, pruebas, producción

ORGANIZACIÓN

Estructura configuraciones grandes en **partes manejables**: un módulo para VMs, otro para redes...

ESTANDARIZACIÓN

Garantiza que recursos similares se creen siempre de forma **coherente y predecible**

Módulos: analogía con funciones

FUNCIÓN EN PROGRAMACIÓN

```
def crear_vm(nombre, memoria, imagen):  
    # lógica interna  
    return ip_asignada
```

- **Parámetros** → valores de entrada
- **Return** → valores de retorno
- **Cuerpo** → lógica interna

MÓDULO EN OPENTOFU

```
module "server" {  
    source      = "../..//terraform/modules/vm"  
    name        = "servidor1"  
    memory      = 1024  
    base_image  = "debian13-base.qcow2"  
}
```

- **Variables** → valores de entrada
- **Outputs** → valores de retorno
- **Resources internos** → lógica interna

Ejemplo: escenario con módulos (escenario.tf)

```
locals {
  networks = {
    red-nat = { mode = "nat", addresses = ["192.168.100.0/24"], dhcp = true }
    red-aislada = { mode = "none", addresses = ["10.0.0.0/24"], dhcp = false }
  }
  servers = {
    server1 = {
      name = "server1", memory = 1024, vcpu = 1
      base_image = "debian13-base.qcow2"
      user_data = "cloud-init/server1/user-data.yaml"
      network_config = "cloud-init/server1/network-config.yaml"
      network = { network_name = "red-nat", wait_for_lease = true }
    }
  }
}
```

 Con módulos, añadir una nueva máquina solo requiere **una entrada más** en `local.servers` — sin crear nuevos ficheros `mainX.tf`.

Sin módulos vs. con módulos

SIN MÓDULOS — ESCENARIO CON 3 VMS

- Crear `main1.tf`, `main2.tf`, `main3.tf`
- Crear `cloud-init/server1/`, `cloud-init/server2/`, `cloud-init/server3/`
- Modificar `output.tf` para las 3 máquinas

Repetitivo, propenso a errores, difícil de mantener

CON MÓDULOS — ESCENARIO CON 3 VMS

- Añadir 3 entradas en `local.servers` del fichero `escenario.tf`
- Crear `cloud-init/serverX/` para cada máquina

El módulo se encarga del resto de forma automática

PI · OPENTOFU + LIBVIRT

¡Gracias!

OpenTofu + libvirt



José Domingo Muñoz



IES Gonzalo Nazareno · Dos Hermanas



PI